

PROVIDER_UPDATE_SUMMARY

Provider Feature Updates Summary

This document summarizes the changes made to each provider to support prompt caching, reasoning, and token budgets.

Summary of Changes

Core Infrastructure (✅ Completed)

- LLMRequest Structure** (`provider.go`)
 - Added Reasoning `*ReasoningConfig` for reasoning/thinking configuration
 - Added CacheConfig `*CacheConfig` for prompt caching configuration
 - Added TokenBudget `*TokenBudget` for budget limits
 - Added ThinkingBudget `int` for per-request thinking token allocation
 - Added SessionID `string` for session-based tracking
- ProviderManager** (`provider.go`)
 - Added tokenTracker `*TokenTracker` for token/cost tracking
 - Added cacheMetrics `*CacheMetrics` for caching performance metrics
 - Enhanced `Generate()` method with:
 - Automatic reasoning config detection for known models
 - Default cache config application
 - Pre-request budget checking
 - Post-request usage tracking
 - Cache metrics collection
 - Added methods:
 - `NewProviderManagerWithBudget()` for custom budgets
 - `GetTokenTracker()` to access token tracking
 - `GetCacheMetrics()` to access cache stats
 - `GetSessionUsage()` for session statistics
 - `GetBudgetStatus()` for budget monitoring
 - `ResetSession()` to clear session data
- Supporting Infrastructure**
 - `reasoning.go`: Comprehensive reasoning support for all model types
 - `cache_control.go`: Caching strategies and cost calculation
 - `token_budget.go`: Token/cost tracking with rate limiting

Provider-Specific Changes

1. Anthropic Provider (✅ Fully Implemented)

File: `anthropic_provider.go`

Status: ✅ Complete - All features fully implemented

Features: - ✅ Prompt caching with `cache_control` directives - ✅ Extended thinking mode with token budgets - ✅ Token budget integration - ✅ Auto-detection of reasoning models - ✅ Configurable cache strategies

Key Changes:

```
// buildRequest() now supports:
// 1. CacheConfig-based caching strategy
cacheConfig := request.CacheConfig
if cacheConfig == nil {
    defaultCache := DefaultCacheConfig()
    cacheConfig = &defaultCache
}

// 2. Reasoning configuration with auto-detection
reasoningConfig := request.Reasoning
if reasoningConfig == nil {
    isReasoning, modelType := IsReasoningModel(request.Model)
    if isReasoning {
        reasoningConfig = NewReasoningConfig(modelType)
    }
}

// 3. Thinking budget from config or request
if reasoningConfig != nil && reasoningConfig.Enabled {
    thinkingBudget := reasoningConfig.ThinkingBudget
    if request.ThinkingBudget > 0 {
        thinkingBudget = request.ThinkingBudget
    }
    req.Thinking = &anthropicThinkingConfig{
        Type: "enabled",
        Budget: thinkingBudget,
    }
}
```

Cache Support: - System messages: Cached with `cache_control: {type: "ephemeral"}` - Tool definitions: Last tool cached for efficiency - Messages: Last message cached in Context/Aggressive strategies - TTL: 5 minutes default

Reasoning Support: - Claude 4 models: Full extended thinking - Claude 3.7 Sonnet: Enhanced reasoning - Auto-detection via model name or keywords - Configurable thinking budgets

2. OpenAI Provider (⚠️ Partial Implementation)

File: `openai_provider.go`

Status: ⚠️ Needs Updates

Required Changes:

1. Add reasoning support for O-series models:

```
func (op *OpenAIProvider) convertToOpenAIRequest(request
    *LLMRequest) (*openaiRequest, error) {
    req := &openaiRequest{
```

```

        Model: request.Model,
        Messages: convertMessages(request.Messages),
        MaxTokens: request.MaxTokens,
        Temperature: request.Temperature,
    }

    // Check if this is a reasoning model (o1, o3, o4)
    if isReasoning, modelType :=
        IsReasoningModel(request.Model); isReasoning {
        // O-series models handle reasoning automatically
        // May need to adjust parameters (o1 doesn't support
        // temperature, etc.)
        switch modelType {
        case ReasoningModelOpenAI_01, ReasoningModelOpenAI_03,
            ReasoningModelOpenAI_04:
            req.Temperature = 0 // O-series uses fixed
            temperature
            req.TopP = 0 // O-series doesn't support top_p
        }
    }

    return req, nil
}

```

1. Add token budget integration:

```

func (op *OpenAIProvider) Generate(ctx context.Context, request
    *LLMRequest) (*LLMResponse, error) {
    // Token budgets are handled by ProviderManager
    // Just need to return accurate usage in response
    response, err := op.makeOpenAIRequest(ctx, openaiRequest)
    if err != nil {
        return nil, err
    }

    // For o-series models, separate thinking tokens
    llmResponse := op.convertFromOpenAIResponse(response,
        request.ID, processingTime)
    if response.Usage.CompletionTokensDetails != nil {
        // Add reasoning tokens to metadata
        llmResponse.ProviderMetadata = map[string]interface{}{
            "reasoning_tokens":
                response.Usage.CompletionTokensDetails.ReasoningTokens,
        }
    }
}

```

```

    return llmResponse, nil
}

```

1. Add o-series models to model list:

```

{
    Name: "o1-preview",
    Provider: ProviderTypeOpenAI,
    ContextSize: 128000,
    MaxTokens: 32768,
    Capabilities: reasoningCapabilities,
    SupportsTools: false, // o1 doesn't support tools yet
    SupportsVision: false,
    Description: "OpenAI O1 Preview - Advanced reasoning model",
},
{
    Name: "o1-mini",
    Provider: ProviderTypeOpenAI,
    ContextSize: 128000,
    MaxTokens: 65536,
    Capabilities: reasoningCapabilities,
    SupportsTools: false,
    SupportsVision: false,
    Description: "OpenAI O1 Mini - Fast reasoning model",
},

```

Priority: High - O-series models are production-ready

3. Azure Provider (⚠ Needs Implementation)

File: azure_provider.go

Status: ⚠ Needs Updates

Required Changes: - Same as OpenAI provider (Azure uses OpenAI API) - Support Azure-specific authentication (Entra ID, API keys) - Handle Azure-specific endpoints - Map Azure deployment names to model capabilities

Implementation:

```

func (ap *AzureProvider) Generate(ctx context.Context, request
    *LLMRequest) (*LLMResponse, error) {
    // Translate Azure deployment to base model
    baseModel := ap.getBaseModel(request.Model)

    // Check for reasoning models
    if isReasoning, modelType := IsReasoningModel(baseModel);
        isReasoning {
        // Handle o-series specific configuration
        // Apply reasoning config
    }
}

```

```

    // Proceed with Azure API call
    return ap.makeAzureRequest(ctx, azureRequest)
}

```

Priority: High - Azure is widely used in enterprise

4. Gemini Provider (⚠ Needs Implementation)

File: gemini_provider.go

Status: ⚠ Needs Updates

Required Changes:

1. Add Gemini 2.0 models with reasoning:

```

{
    Name: "gemini-2.0-flash-thinking-exp",
    Provider: ProviderTypeGemini,
    ContextSize: 1000000,
    MaxTokens: 8192,
    Capabilities: allCapabilities,
    SupportsTools: true,
    SupportsVision: true,
    Description: "Gemini 2.0 Flash with thinking mode
                 (experimental)",
},

```

1. Support thinking mode:

```

func (gp *GeminiProvider) buildRequest(request *LLMRequest)
(*geminiRequest, error) {
    req := &geminiRequest{
        Contents: gp.convertMessages(request.Messages),
        GenerationConfig: geminiGenerationConfig{
            MaxOutputTokens: request.MaxTokens,
            Temperature: request.Temperature,
        },
    }

    // Check for thinking models (gemini-2.0-flash-thinking-exp)
    if strings.Contains(request.Model, "thinking") {
        if request.Reasoning != nil && request.Reasoning.Enabled
        {
            req.GenerationConfig.ThinkingConfig =
            &geminiThinkingConfig{
                Enabled: true,
            }
        }
    }
}

```

```

    }

    return req, nil
}

```

1. Add context caching support:

```

// Gemini uses cachedContent API
// Need to create cached content first, then reference in
    requests

```

Priority: Medium - Gemini 2.0 thinking is experimental

5. VertexAI Provider (⚠ Needs Implementation)

File: vertexai_provider.go

Status: ⚠ Needs Updates

Required Changes: - Similar to Gemini provider (VertexAI hosts Gemini models) - Add GCP authentication and project configuration - Support Vertex-specific features (tuned models, etc.)

Priority: Medium

6. Bedrock Provider (⚠ Needs Implementation)

File: bedrock_provider.go

Status: ⚠ Needs Updates

Required Changes:

1. Support Claude models on Bedrock:

```

func (bp *BedrockProvider) Generate(ctx context.Context, request
    *LLMRequest) (*LLMResponse, error) {
    // Detect underlying model (Claude, etc.)
    if strings.Contains(request.Model, "claude") {
        // Use Anthropic-style features via Bedrock API
        // Bedrock supports prompt caching for Claude
        // Extended thinking may be available
    }

    return bp.invokeModel(ctx, bedrockRequest)
}

```

1. Add Bedrock-specific caching:

```

// AWS Bedrock supports prompt caching for Claude models
// Use AWS SDK to enable caching

```

Priority: Medium - Bedrock is popular for enterprise AWS users

7. Groq Provider (✅ Partial - Budget Only)

File: groq_provider.go

Status: ✅ Token budgets supported, reasoning model-dependent

Changes Needed: - Token budgets: ✅ Already supported via ProviderManager - Caching: ❌ Not supported by Groq API - Reasoning: ⚠️ Depends on model (DeepSeek-R1, etc.)

Implementation:

```
// Reasoning support for compatible models
if strings.Contains(request.Model, "deepseek") &&
    strings.Contains(request.Model, "r1") {
    // Apply DeepSeek R1 reasoning configuration
    reasoningConfig :=
        llm.NewReasoningConfig(llm.ReasoningModelDeepSeek_R1)
}
```

Priority: Low - Limited feature support

8. Qwen Provider (⚠️ Needs Implementation)

File: qwen_provider.go

Status: ⚠️ Needs Updates

Required Changes:

1. Add QwQ-32B reasoning model:

```
{
    Name: "qwq-32b-preview",
    Provider: ProviderTypeQwen,
    ContextSize: 32000,
    MaxTokens: 8192,
    Capabilities: allCapabilities,
    SupportsTools: true,
    SupportsVision: false,
    Description: "QwQ-32B Preview - Reasoning model",
},
```

1. Support thinking mode:

```
func (qp *QwenProvider) buildRequest(request *LLMRequest)
    (*qwenRequest, error) {
    // Check for QwQ-32B
    if strings.Contains(request.Model, "qwq") {
        if request.Reasoning != nil && request.Reasoning.Enabled
        {
            // QwQ uses thinking tags in prompt
            prompt := FormatReasoningPrompt(
```

```

        request.Messages[len(request.Messages)-1].Content,
        request.Reasoning,
    )
    // Update last message with formatted prompt
}
}

return req, nil
}

```

Priority: Low-Medium

9. xAI Provider (⚠ Needs Implementation)

File: xai_provider.go

Status: ⚠ Needs Updates

Required Changes: - Token budgets: ✅ Supported via ProviderManager - Reasoning: Add Grok model reasoning support - Caching: ❌ Not supported by xAI API

Priority: Low

10. OpenRouter Provider (✅ Passthrough)

File: openrouter_provider.go

Status: ✅ Features passed through to underlying models

Implementation: - Token budgets: ✅ Supported via ProviderManager - Reasoning: ✅ Depends on selected model (o1, Claude, etc.) - Caching: ⚠ Depends on upstream provider

Notes: - OpenRouter routes to multiple providers - Features depend on selected model - ProviderManager automatically handles this

Priority: Low - Already functional

11. Copilot Provider (⚠ Needs Implementation)

File: copilot_provider.go

Status: ⚠ Needs Updates

Required Changes: - Similar to OpenAI (Copilot uses GPT-4 based models) - GitHub-specific authentication - Token budgets via ProviderManager

Priority: Low

12. Local Provider (Ollama, llama.cpp) (⚠ Needs Implementation)

File: local_provider.go, ollama_provider.go, llamacpp_provider.go

Status: ⚠ Needs Updates

Required Changes:

1. Support local reasoning models:

```
// Models like QwQ-32B, DeepSeek-R1 can run locally
func (lp *LocalProvider) Generate(ctx context.Context, request
    *LLMRequest) (*LLMResponse, error) {
    // Check for reasoning models
    if isReasoning, modelType :=
        IsReasoningModel(request.Model); isReasoning {
        // Format prompt for reasoning
        if request.Reasoning != nil && request.Reasoning.Enabled
        {
            prompt := FormatReasoningPrompt(

                request.Messages[len(request.Messages)-1].Content,
                request.Reasoning,
            )
            // Use formatted prompt with thinking instructions
        }
    }

    return lp.callLocalModel(ctx, localRequest)
}
```

1. Token budgets:

- ✅ Already supported via ProviderManager
- Local models don't have API costs, but still useful for resource limits

Priority: Medium - Many users run local models

Implementation Priorities

High Priority (Critical)

1. ✅ Core infrastructure (ProviderManager, LLMRequest)
2. ✅ Anthropic provider (complete)
3. ⚠ OpenAI provider (o-series reasoning)
4. ⚠ Azure provider (enterprise use)

Medium Priority (Important)

1. ⚠ Bedrock provider (AWS enterprise)
2. ⚠ Gemini/VertexAI providers (Google ecosystem)
3. ⚠ Local providers (cost-effective, privacy)

Low Priority (Nice to Have)

1. ⚠️ Qwen provider (QwQ-32B)
2. ⚠️ Groq provider (fast inference)
3. ⚠️ xAI provider (Grok models)
4. ⚠️ Copilot provider (IDE integration)
5. ✅ OpenRouter (passthrough, already works)

Testing Strategy

For each provider, add tests for:

1. Reasoning Support:

```
func TestProviderReasoning(t *testing.T) {
    request := &llm.LLMRequest{
        Model: "reasoning-model-name",
        Messages: []llm.Message{
            {Role: "user", Content: "Explain why 2+2=4 step by step"},
        },
        Reasoning:
            llm.NewReasoningConfig(llm.ReasoningModelType),
    }

    response, err := provider.Generate(ctx, request)
    assert.NoError(t, err)
    assert.NotEmpty(t, response.Content)

    // For providers that support thinking tokens
    if metadata, ok := response.ProviderMetadata.
        (map[string]interface{}); ok {
        assert.Greater(t, metadata["thinking_tokens"], 0)
    }
}
```

1. Caching Support:

```
func TestProviderCaching(t *testing.T) {
    cacheConfig := &llm.CacheConfig{
        Enabled: true,
        Strategy: llm.CacheStrategyTools,
    }

    // First request (creates cache)
    response1, err := provider.Generate(ctx, &llm.LLMRequest{
        Model: "model-name",
        Messages: messages,
        CacheConfig: cacheConfig,
    })
}
```

```

// Second request (uses cache)
response2, err := provider.Generate(ctx, &llm.LLMRequest{
    Model: "model-name",
    Messages: messages,
    CacheConfig: cacheConfig,
})

// Verify cache was used (if supported)
if metadata, ok := response2.ProviderMetadata.
    (map[string]interface{}); ok {
    assert.Greater(t, metadata["cache_read_tokens"], 0)
}
}

```

1. Token Budgets:

```

func TestTokenBudgets(t *testing.T) {
    budget := llm.TokenBudget{
        MaxTokensPerRequest: 1000,
        MaxTokensPerSession: 5000,
    }

    pm := llm.NewProviderManagerWithBudget(config, budget)
    sessionID := "test-session"

    // Make requests until budget is approached
    for i := 0; i < 6; i++ {
        _, err := pm.Generate(ctx, &llm.LLMRequest{
            Model: "model-name",
            Messages: messages,
            SessionID: sessionID,
        })

        if i < 5 {
            assert.NoError(t, err) // Should succeed
        } else {
            assert.Error(t, err) // Should fail on budget
        }
    }
}

```

Documentation Updates

All providers now have: -  Feature matrix documentation (PROVIDER_FEATURES.md) - 
 Implementation guide for each feature -  Code examples and configuration samples -  Cost
 estimation and optimization tips -  Migration guide for existing code

Backward Compatibility

All changes maintain backward compatibility: -  Existing `LLMRequest` usage continues to work -  New fields are optional (`nil` = disabled) -  Providers without feature support silently ignore configs -  Default configs can be applied automatically

Next Steps

1.  Implement high-priority providers (OpenAI, Azure)
2.  Add comprehensive tests for all providers
3.  Create provider-specific documentation
4.  Add integration examples
5.  Performance benchmarking
6.  Cost optimization analysis

Conclusion

The core infrastructure is complete and production-ready. Provider-specific implementations can be rolled out incrementally based on priority and user demand. The Anthropic provider serves as a reference implementation for other providers to follow.

Sources verified

Sources verified 2026-05-29: <https://platform.claude.com/docs/en/docs/about-claude/models/overview> (current Anthropic Claude model IDs — `claude-sonnet-4-6`, `claude-haiku-4-5`, plus the Opus family; older API aliases such as `claude-3-5-sonnet-latest` are no longer in the current/legacy model table), <https://go.dev/doc/devel/release> (Go 1.26.3 latest patch — matches `CLAUDE.md` §3.1). Confirmed: this summary documents provider-integration code-structure (request/response conversion, reasoning-model handling, capability flags), which per `CONST-036` must source live model metadata from `LLMsVerifier` rather than from any hardcoded list, so the document is not itself an operator-facing model catalogue. Negative finding: the inline `Description` strings and reasoning-model constant names (e.g. `ReasoningModelOpenAI_01/03/04`) are illustrative of historical model generations and are NOT the runtime source of truth — the live provider/model set is verifier-driven; per §11.4.6 no model IDs were invented here, and stale-looking literals are left as code-illustration rather than rewritten to guessed current IDs.